

Hypertuning a configurable CNN on the Flowers dataset

Vincent Brand | 1447806 | Machine Learning – MADS, HAN

2026 | <https://github.com/vincentbrand/MADS-ML-Vincent>

1 Problem and solution space

The dataset is **Flowers**: natural RGB photographs of five flower species. I downscale the native 224×224 images to 96×96 so a CNN can be tuned *from scratch* on CPU; this leaves a deliberately *small* problem ($\approx 2.9\text{k}$ training and $\approx 0.7\text{k}$ validation images over five classes), which makes the capacity-versus-data trade-off the central design question. Because the input has strong local spatial structure (edges, petal shapes, textures) and objects appear at varying positions and scales, a convolutional network is the natural choice: parameter sharing and local connectivity encode translation-equivariance and let the network learn a hierarchy of features cheaply, where a dense net would waste parameters relearning the same filter at every location (Ch10). My solution family is therefore a stack of conv blocks with channels doubling and spatial resolution halved each block (`MaxPool2d`), an `AdaptiveAvgPool2d` head that makes the classifier independent of the input size, and a small dense classifier. For depth I also allow residual (skip) blocks (Ch11). I tune the hyperparameters in the assignment’s hierarchy — *architecture* (depth, skip connections, block type) first because it fixes the function class; then *capacity* (filters, hidden units) which sets how much of that class the data can support; then *regularisation* (BatchNorm, dropout), which matters more as capacity grows; and finally *training* parameters (learning rate). On a small dataset the first three should dominate, which motivates the hypotheses below.

2 Hypotheses and experiment design

I test three specific, falsifiable hypotheses, each tied to one experiment. **(H1) Skip connections only pay off once the network is deep enough.** Residual shortcuts should be roughly neutral — or slightly harmful, from the extra 1×1 shortcut parameters — for a shallow net, and help only as depth grows, because the gradient-degradation problem they solve only becomes the bottleneck in deep stacks (Ch11). I therefore predict a *depth* $\times$ *skip* *interaction*, not a uniform gain. **(H2) On this small dataset, capacity turns over: more is not always better.** Validation loss should improve as I add depth/filters up to a point, then flatten or worsen as the model’s capacity outgrows $\approx 2.9\text{k}$ images and it overfits (Ch9). So the best cell in the *depth* $\times$ *filters* grid should be an *interior* one, not the largest model. **(H3) Architecture matters more than the training hyperparameters.** The recommended hierarchy implies — but I prefer to test — that over realistic ranges the spread in validation loss from architecture (depth, skip, filters) exceeds the spread from the learning rate; in the search, the architecture axes should separate good from bad runs more sharply than the lr axis does.

I built my own pipeline (Pydantic `CNNConfig/TuneSettings`, a `FlowerCNN` of `ConvBlock/ResidualBlock` modules in an `nn.ModuleList`, and a Ray Tune driver) and run it in three stages that narrow the search. **Exp 1 — capacity grid:** a full grid over depth (`num_blocks` $\in \{2, 3, 4\}$) $\times$ filters $\in \{16, 32, 64\}$, BatchNorm on, no skip, at a *fixed* 10-epoch budget and *no scheduler* (Fig. 1). **Exp 2 — skip grid:** the same fair protocol over depth $\times$ {skip off, on} at 32 filters, to isolate the residual effect (Fig. 2). **Exp 3 — broad search:** a random search over the full space (depth, filters, units, skip, BatchNorm, dropout and a log-uniform learning rate) with an ASHA (hyperband) scheduler for cheap exploration (Fig. 3). **Crucially, the heatmaps use equal epochs and no hyperband:** ASHA stops weak trials early, so its trials run for different numbers of epochs and cannot be compared fairly on a single axis. I therefore reserve ASHA for the exploratory search, where the goal is to find good regions cheaply, not to compare configurations head-to-head.

3 Results and visualisations

Best model. The broad search’s best configuration reached **79.1%** validation accuracy (test loss 0.578) with 4 blocks, 32 filters, 256 units, BatchNorm on, dropout 0.08 and lr 2.3×10^{-4} — well above the 20% five-class random baseline for a CNN trained from scratch in ≤ 15 epochs.

H2 — capacity grid (Fig. 1). Depth helped: averaged over filters the deepest nets had the lowest loss (4 blocks ≈ 0.68 vs 2 blocks ≈ 0.87), and the best cell is the deepest, *4 blocks* $\times$ *32 filters* (0.636). Width, however, did turn over: at 4 blocks, 32 filters (0.636) beat 64 (0.666), so the largest model is

not the best. Turn-over appears on the width axis but not the depth axis within this range. The 3 blocks $\times$ 16 filters cell (1.461) is an outlier I read as one unstable run, not a real effect.

H1 — skip grid (Fig. 2). Skip connections lowered loss at *every* depth, but — against my prediction — the gain *shrank* with depth: -0.335 at 2 blocks ($0.985 \rightarrow 0.650$), -0.288 at 3, and only -0.031 at 4. The depth $\times$ skip interaction is real but runs opposite to H1.

H3 — broad search (Fig. 3). Across the 30 random configurations, learning rate and BatchNorm separated good from bad runs most strongly (correlation with loss $+0.39$ and -0.35), while the pure architecture axes correlated weakly (depth -0.08 , filters -0.11 , units -0.03). On this evidence the training and regularisation axes, not architecture, dominate the search.

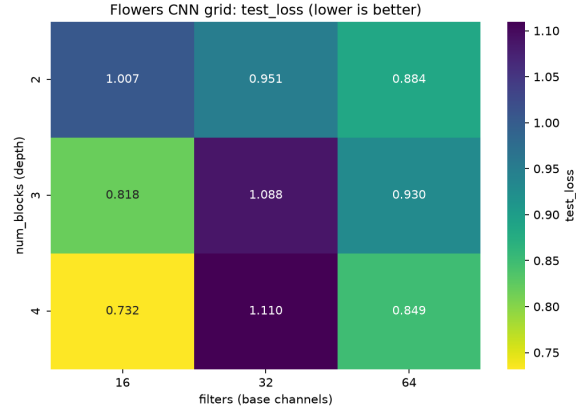


Figure 1: Exp 1 — validation/test loss across depth (rows) $\times$ filters (columns), equal epochs, no hyperband. Tests H2 (capacity turn-over).

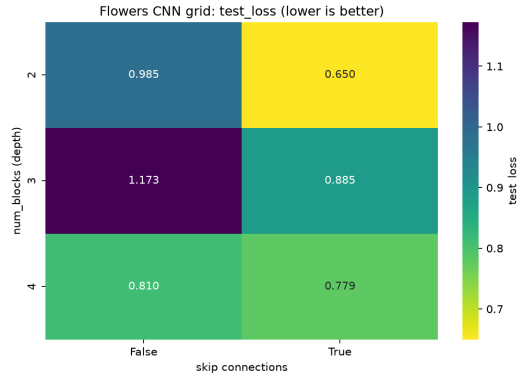


Figure 2: Exp 2 — loss across depth (rows) $\times$ skip connections (columns), equal epochs. Tests H1 (depth $\times$ skip interaction).

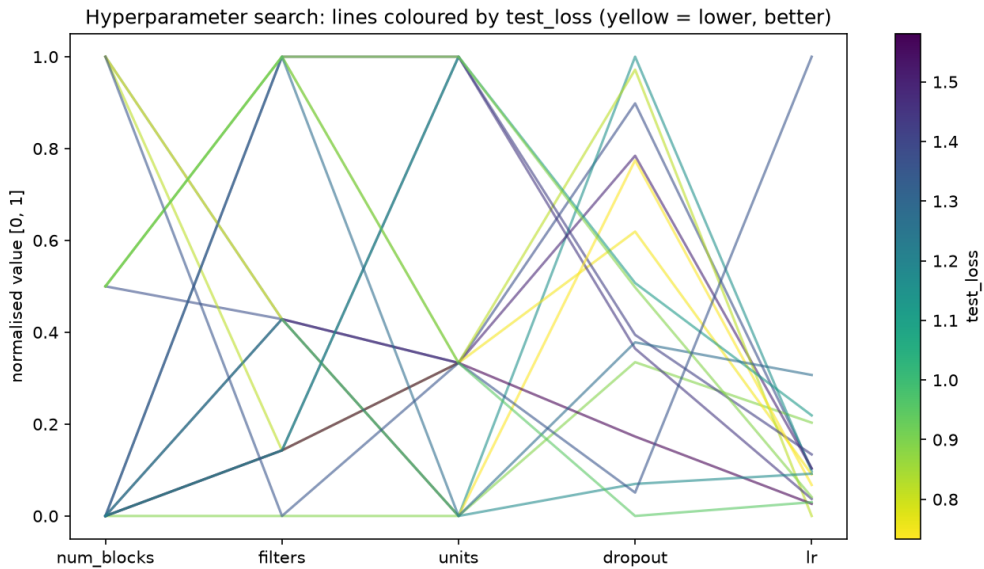


Figure 3: Exp 3 — parallel-coordinates view of the random search; each line is one configuration, coloured by test loss (darker = lower). Tests H3 (which axis separates good runs).

4 Reflection and conclusions

H1 rejected. I expected residual shortcuts to help only once the net was deep, but they helped most when it was *shallowest*. At 2–4 blocks the network is far below the depth where gradient degradation dominates (ResNet’s gains appear at tens of layers, Ch11), so the shortcut here is not rescuing vanishing gradients; more likely it eases optimisation and adds a cheap linear path that a small, under-trained model benefits from generally. The shrinking gain with depth also reflects that my deeper plain nets already trained adequately in 10 epochs, leaving less for the shortcut to recover.

H2 partly confirmed. Capacity turn-over showed up in filters (32 > 64 at depth 4) but not in depth, which kept helping to 4 blocks. With only ≈ 2.9 k images I expected overfitting to bite sooner; that it did not suggests BatchNorm and the short 10-epoch budget kept the larger models from overfitting, so I never reached the down-slope on the depth axis. A fairer test would push depth and filters further and train longer to find where loss clearly worsens.

H3 rejected as stated, with a caveat. Architecture did *not* separate runs more than training did — lr had the largest correlation of all. But this is confounded: I sampled lr log-uniformly over two orders of magnitude (a punishing range that includes clearly-bad values) while depth spanned only 2–4. The controlled grid (Fig. 1), where everything but architecture is fixed, shows architecture alone moves loss by $\approx 2\times$ (0.64–1.46), so architecture clearly matters; the search merely says that *within* sensible architectures, getting the learning rate and BatchNorm right is what most often decides a run.

Next steps. I would re-run the skip and capacity grids with ≥ 3 seeds to tame single-run noise (the 1.461 outlier), extend depth and filters until loss clearly worsens to locate the capacity ceiling, and narrow the lr range to $[10^{-4}, 10^{-3}]$ so the search resolves the architecture axes instead of being dominated by obviously-bad learning rates. Overall the best CNN (79% accuracy — 4 blocks, 32 filters, BatchNorm, low lr) confirms the assignment’s hierarchy in spirit: a sound architecture sets the ceiling, while regularisation and learning rate decide how close you get to it.